

安装部署 Spark 并用虚拟环境运行 PySpark 命令行

前置条件：请在完成 [7.Hadoop伪分布式部署](#) 之后，再来完成本文档的内容。

💡 实验必备：理解 Python 虚拟环境 (Virtual Environment)

在进行"简历-岗位匹配"这样的大数据 AI 项目时，我们不仅要写代码，还要安装大量的依赖库（如 PySpark、jieba、Scikit-learn）。为了管理这些复杂的库，**虚拟环境**是每位开发者的"标配"。

1. 什么是虚拟环境？

你可以把虚拟环境想象成一个**"隔离的实验沙盒"**。

- **物理本质：**它其实就是你家目录下（如 `~/ai_env`）的一个普通文件夹，里面存放了一套独立的 Python 解释器和一份私有的插件库清单。
- **形象比喻：**系统自带的 Python 像是"公共大食堂"，所有人都在里面用同样的调料；而虚拟环境则是你的"私人小厨房"，你可以根据项目需求随意安装特定版本的插件，而不会影响到系统的稳定性。

2. 为什么本次实训"必须"使用它？

- **绕过系统限制（针对 Ubuntu 24.04）：**从 Ubuntu 23.04 开始，系统为了防止用户乱装插件导致桌面环境崩溃，默认禁止使用 `pip install` 向全局环境安装库（报错：`externally-managed-environment`）。**使用虚拟环境是目前在 Linux 上安装 Python 库的唯一官方标准做法。**
- **版本"零冲突"：**实训要求 PySpark 版本必须是 3.5.1。如果你的系统中其他软件需要旧版 PySpark，两者就会打架。虚拟环境能让本项目拥有完全独立的"生存空间"。

- **一键打包与分发**：当老师把虚拟机镜像发给学生，或者学生把代码交给老师评分时，虚拟环境确保了“**环境一致性**”。只要虚拟环境文件夹在，代码在哪里跑出来的结果都一模一样，避开了“在我电脑上能跑，在你那里就不行”的玄学问题。

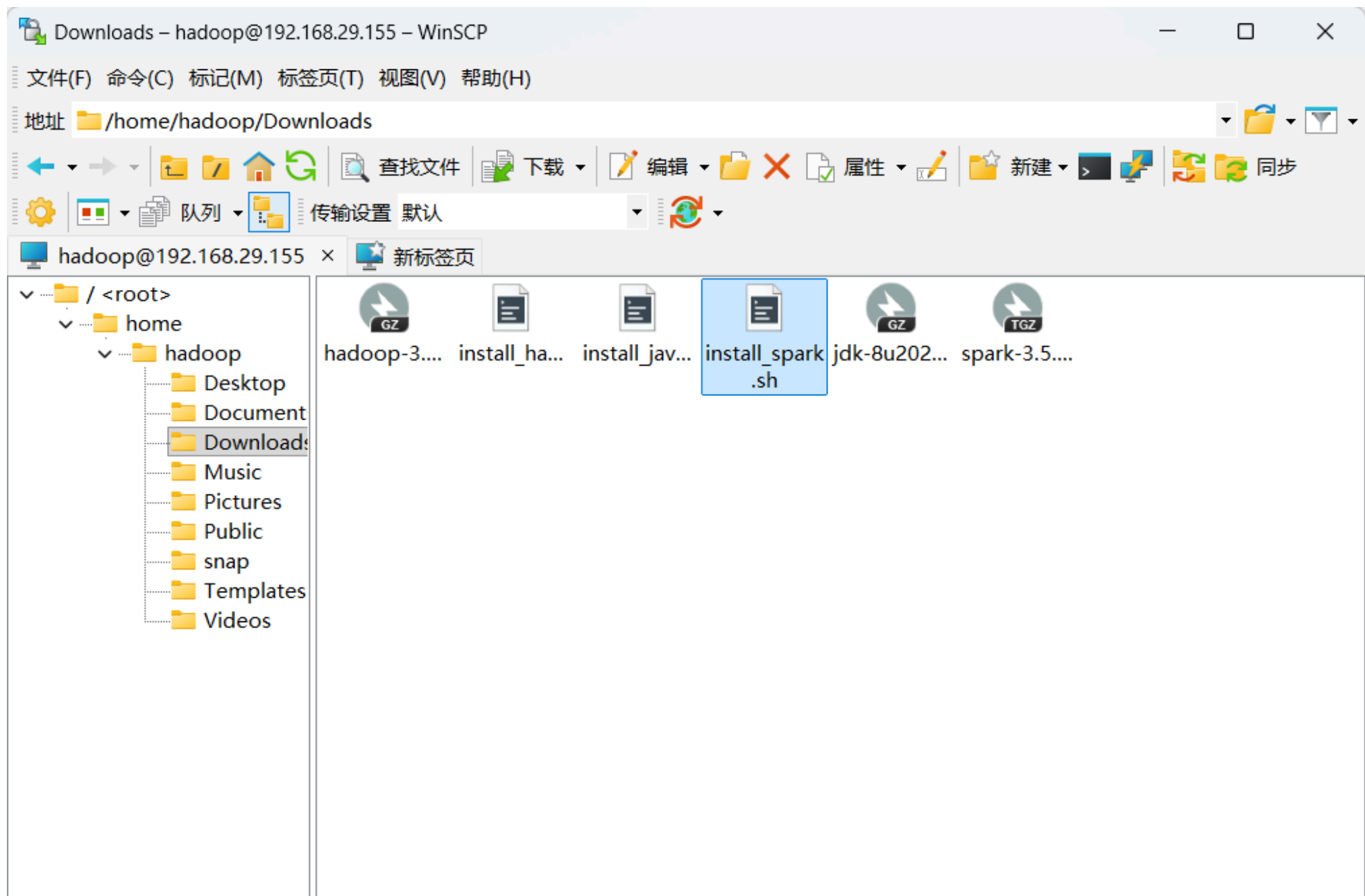
3. 核心操作小贴士

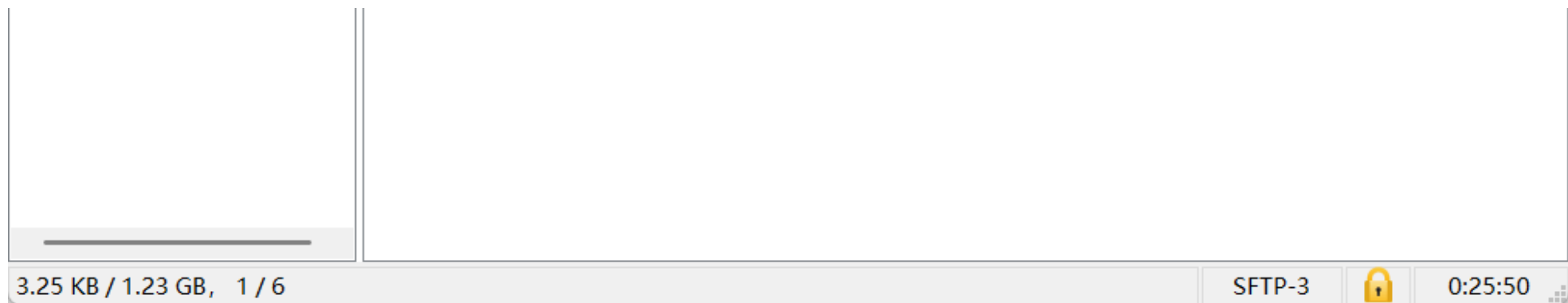
- **激活（进入厨房）**：`source ~/ai_env/bin/activate`。激活后，你会看到命令行开头多了个 `(ai_env)` 标记。
- **退出（关门下班）**：输入 `deactivate` 即可回到系统默认环境。

⚠ 老师提醒：在 PyCharm 中编写代码前，**第一件事**就是把项目的“Python Interpreter（解析器）”指向这个虚拟环境文件夹。记住：**先有环境，后写代码**。

步骤 1：上传 Spark 安装包

使用 WinSCP 将 Spark 安装包和安装脚本传到虚拟机当中。





步骤 2：执行安装脚本

打开终端，输入命令运行安装脚本安装 Spark 并创建专用的 Python 虚拟环境：

```
cd ~/Downloads  
chmod +x install_spark.sh  
./install_spark.sh
```

```
hadoop@hadoop: ~/Downloads
hadoop@hadoop:~/Desktop$ cd ~/Downloads
hadoop@hadoop:~/Downloads$ chmod +x install_spark.sh
hadoop@hadoop:~/Downloads$ ./install_spark.sh
=== Spark 3.5.1 自动化安装启动 ===
[检查通过] 检测到安装包: spark-3.5.1-bin-hadoop3.tgz
[检查通过] 检测到 Hadoop 路径: /usr/local/hadoop
正在准备安装目录: /usr/local/spark
[sudo] hadoop 的密码:
正在解压并平铺 Spark 文件...
[成功] Spark 已安装至 /usr/local/spark
正在配置系统环境变量...
[配置成功] SPARK_HOME 已加入 ~/.bashrc
正在配置 spark-env.sh...
正在为实训创建 Python 虚拟环境 (ai_env)...

WARNING: apt does not have a stable CLI interface. Use with caution in scripts.

[成功] 虚拟环境 ai_env 已创建在用户家目录。
=====
Spark 安装及 HDFS 联动配置完成!
请执行以下步骤激活环境:
1. 执行: source ~/.bashrc
2. 激活 Python 环境: source ~/ai_env/bin/activate
```

```
3. 安装 PySpark 库: pip install pyspark==3.5.1 jieba pandas scikit-learn streaml
```

步骤 3：激活虚拟环境并安装依赖库

打开另一个终端窗口，激活 Python 环境，激活后，命令行前面会有个括号，括号里显示虚拟环境的名称。然后连接阿里云 PyPI 镜像安装 pyspark 3.5.1、jieba、pandas、scikit-learn、streamlit 库。

```
source ~/ai_env/bin/activate  
pip install pyspark==3.5.1 jieba pandas scikit-learn streamlit -i https://mirrors.aliyun.com/pypi/simple/ --trusted-host  
mirrors.aliyun.com
```

```
hadoop@hadoop: ~/Desktop
hadoop@hadoop:~/Desktop$ source ~/ai_env/bin/activate
(ai_env) hadoop@hadoop:~/Desktop$ pip install pyspark==3.5.1 jieba pandas scikit-learn streamlit -i https://mirrors.aliyun.com/pypi/simple/ --trusted-host mirrors.aliyun.com
Looking in indexes: https://mirrors.aliyun.com/pypi/simple/
Collecting pyspark==3.5.1
  Downloading https://mirrors.aliyun.com/pypi/packages/73/e5/c9eb78cc982dafb7b5834bc5c368fe596216c8b9f7c4b4ffa104c4d2ab8f/pyspark-3.5.1.tar.gz (317.0 MB)
    317.0/317.0 MB 697.1 kB/s eta 0:00:00
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Collecting jieba
  Downloading https://mirrors.aliyun.com/pypi/packages/c6/cb/18eeb235f833b726522d7ebed54f2278ce28ba9438e3135ab0278d9792a2/jieba-0.42.1.tar.gz (19.2 MB)
    19.2/19.2 MB 684.6 kB/s eta 0:00:00
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Collecting pandas
  Downloading https://mirrors.aliyun.com/pypi/packages/65/b6/09b01cdbc15224e2850365192d17b7bdebb8bdbc8780ed221fcdf0d9a515/pandas-3.0.3-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (10.9 MB)
```

10.9/10.9 MB 650.6 kB/s eta 0:00:00

步骤 4：验证 PySpark 识别 Spark

输入指令，查看 PySpark 库能否识别到本机安装的 Spark。如果显示结果如下图所示在 `/usr/local/spark` 下，那么说明识别到了本机安装的 Spark；如果显示在虚拟环境目录下，说明调用的是虚拟环境里的 Python 库包，需要检查环境变量优先级。

```
hadoop@hadoop:~/Desktop$ which pyspark
/usr/local/spark/bin/pyspark
hadoop@hadoop:~/Desktop$
```

💡 实验必备：理清 PySpark、Spark 引擎与 Hadoop 的"三角关系"

在动手编写"简历-岗位匹配"代码前，理清这三者的逻辑关系是大数据开发的"基本功"。它们并不是孤立的软件，而是一个高度协同的"分布式生产流水线"。

1. PySpark 与 Spark 引擎：翻译官与核心工厂

- **Spark 引擎** (`/usr/local/spark`)：计算的"核心工厂"，负责内存计算和任务调度。
- **PySpark**：它是工厂派出的"**首席翻译官**"。由于引擎底层听不懂 Python，PySpark 负责将你的 Python 代码翻译成引擎能理解的 JVM 指令。
- **纽带**：环境变量 `SPARK_HOME` 就是翻译官找到工厂的"导航地址"。

2. 命令行操作对比：启动"服务" vs 启动"入口"

操作指令	对应脚本位置	行为本质	形象比喻
启动 Spark 服务	sbin/start-all.sh	启动 Master 和 Worker 常驻后台进程 。	工厂大开门 ：保安（进程）上岗，等待接单。
启动 PySpark 交互	bin/pyspark	启动一个 Python 运行环境（Driver） 。	特约项目经理 ：带上图纸直接进场，干完活就解散。

注意：在实训中，我们通常**不需要**执行 `start-all.sh`，而是直接运行 `pyspark` 或 Python 脚本。

3. 核心解密：没开 Master/Worker，PySpark 怎么算？

启动 `pyspark` 时，**默认不会**启动 Spark 自带的 Master 和 Worker 进程。计算由以下两种机制完成：

- **本地模式（`local[*]`）：全能自由职业者**
 - **原理：**`pyspark` 进程既是指挥官也是工人。它在当前进程内部开启**多个线程**，利用本机多核 CPU 模拟计算。
 - **关系：****完全不依赖** Hadoop。适合在自家里（本地内存）调试小规模匹配算法。
- **集群模式（`yarn`）：临时外包工程**
 - **原理：**Spark 不使用自己的 Master/Worker，而是向 Hadoop 的 **YARN 建筑公司**租场地。
 - **Container 内部细节：**YARN 会动态划出"临时工地"（Container）。
 - **ApplicationMaster (AM)：**租下的第一个工地，里面坐着**项目经理**，负责调度。
 - **Executor：**后续租下的工地，里面坐着**临时工**，负责分词、计算相似度。
 - **关系：****强依赖** Hadoop。活干完后，所有工地（Container）立即退租销毁，**全程无固定常驻进程**。

4. 运行模式对比表（实训决策参考）

模式	核心配置	YARN/HDFS 状态	计算靠谁	内存压力
本地模式	<code>.master("local[*]")</code>	关/开均可	本机 CPU 多线程	较小（推荐）
集群模式	<code>.master("yarn")</code>	必须全部开启	YARN 里的 Container	较大（验收用）

5. 如何"召唤" HDFS：数据导管

为什么代码里写 `hdfs://...` 就能读到数据？

- **核心：** 靠环境变量 `HADOOP_CONF_DIR` 。它指向了 Hadoop 的配置目录。
- **原理：** 它像一根**数据导管**，让 Spark 掌握了 HDFS 仓库的"钥匙"。有了它，无论在何种模式下，Spark 都能直接从分布式仓库中抽取海量简历，实现"**计算向数据移动**"。

 **老师总结：** 在大数据实训中，Hadoop **提供地盘**（YARN）和**原材料**（HDFS），Spark **提供技术**（**计算引擎**），PySpark **提供指令**（**Python 入口**）。在开发阶段，我们先用"本地模式"在自家作坊练兵；在结项阶段，我们再上"YARN 模式"去工业流水线实战。

步骤 5：测试 PySpark 命令行模式

下面测试 PySpark 命令行模式是否可用。输入指令进入 PySpark：

```
pyspark --master local[*]
```

```
(ai_env) hadoop@hadoop:~/Desktop$ pyspark --master local[*]   
Python 3.12.3 (main, Mar 23 2026, 19:04:32) [GCC 13.3.0] on linux  
Type "help", "copyright", "credits" or "license" for more information.  
Setting default log level to "WARN".  
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).  
26/05/28 22:37:30 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable  
Welcome to
```

步骤 6：退出 PySpark 命令行

输入指令退出 PySpark 命令行：

```
exit()
```

```
SparkSession available as 'spark'.  
>>> exit  
Use exit() or Ctrl-D (i.e. EOF) to exit  
>>> exit()  
(ai_env) hadoop@hadoop:~/Desktop$
```

步骤 7：退出虚拟环境

输入指令退出 Python 虚拟环境。确认无误后，请关机并拍摄快照：

```
deactivate
```

```
(ai_env) hadoop@hadoop:~/Desktop$ deactivate  
hadoop@hadoop:~/Desktop$
```